

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf\_a2b97cff-dcd\agent-ac7ff569f92952fb3.jsonl`
- SHA-256: `d3fa3d3a141b53aa0e5a967e6ecab5ceef3f1274db101bb6146cea7e4e141195`
- Source modified: `2026-06-21T13:37:34+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf\_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:34:48.458Z)

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

- pyproject.toml: hatchling build, namespace package (no backend/__init__.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi). The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).

khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.
Always ground claims in file:line or a doc path. Read the actual files; do not assume.

AREA: bundling the web UI. READ: khelsutra/web/ (package.json, vite config, build output dir, src/api client + how it calls the engine /api/*, i18n setup), khelsutra/site/, the engine's backend/api/ (static serving? main.py static mounts?), and confirm whether backend/api/static is dead. Answer: how the SPA is built + served today; the cross-repo coupling (SPA in khelsutra, engine here); options to ship the BUILT SPA inside the Python app (vendored static dir served by FastAPI = unified origin); what the SPA hardcodes about the API base URL; how en/kn/hi catalogs ship.

2. assistant (2026-06-21T13:34:51.199Z)

I'll investigate how the web UI is built and served, and the cross-repo coupling. Let me start by pulling the relevant repos and reading the key files.

3. user (2026-06-21T13:34:55.076Z)

```
Already up to date.
---STATUS---
?? .claude/
?? site/access.db
---BRANCH---
docs/dry-run-pathb-done
```

4. user (2026-06-21T13:34:55.306Z)

```
.
./.claude
./.git
./.github
./.github/workflows
./docs
./node_modules
./site
./site/node_modules
./site/public
./site/scripts
./site/src
./web
./web/dist
./web/node_modules
./web/src
```

5. assistant (2026-06-21T13:34:58.394Z)

Let me read the key files for the web SPA build and API client.

6. user (2026-06-21T13:34:59.351Z)

```
1      {
2        "name": "khelsutra-web",
3        "private": true,
4        "version": "0.0.0",
5        "type": "module",
6        "description": "KhelSutra Studio ? operator console SPA (drives the rally indexer
engine).",
7        "scripts": {
8          "dev": "vite",
9          "build": "vite build",
10         "preview": "vite preview",
11         "typecheck": "tsc --noEmit",
12         "test": "vitest run"
13       },
14       "dependencies": {
15         "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
16         "@fontsource-variable/noto-sans-kannada": "^5.2.9",
```

```

17     "i18next": "^26.3.1",
18     "i18next-browser-languagedetector": "^8.2.1",
19     "i18next-icu": "^2.4.3",
20     "react": "^19.0.0",
21     "react-dom": "^19.0.0",
22     "react-i18next": "^17.0.8"
23   },
24   "devDependencies": {
25     "@tailwindcss/vite": "^4.0.0",
26     "@testing-library/dom": "^10.4.1",
27     "@testing-library/react": "^16.3.2",
28     "@types/node": "^26.0.0",
29     "@types/react": "^19.0.0",
30     "@types/react-dom": "^19.0.0",
31     "@vitejs/plugin-react": "^4.3.4",
32     "jsdom": "^29.1.1",
33     "tailwindcss": "^4.0.0",
34     "typescript": "^5.7.0",
35     "vite": "^6.0.0",
36     "vitest": "^3.0.0"
37   }
38 }
39

```

7. user (2026-06-21T13:34:59.849Z)

No files found

8. user (2026-06-21T13:35:03.278Z)

No files found

9. user (2026-06-21T13:35:04.411Z)

```

total 195
drwxr-xr-x 1 avidu 197609      0 Jun 21 12:31 .
drwxr-xr-x 1 avidu 197609      0 Jun 21 15:38 ..
-rw-r--r-- 1 avidu 197609    41 Jun 20 15:25 .gitignore
-rw-r--r-- 1 avidu 197609  1427 Jun 20 15:25 README.md
drwxr-xr-x 1 avidu 197609      0 Jun 21 15:41 dist
-rw-r--r-- 1 avidu 197609   303 Jun 20 15:25 index.html
-rw-r--r-- 1 avidu 197609 127996 Jun 21 12:31 package-lock.json
-rw-r--r-- 1 avidu 197609   1074 Jun 21 12:31 package.json
drwxr-xr-x 1 avidu 197609      0 Jun 21 15:45 src
-rw-r--r-- 1 avidu 197609   599 Jun 21 12:25 tsconfig.json
-rw-r--r-- 1 avidu 197609   1070 Jun 21 12:12 vite.config.ts
---SRC---
src/api/client.test.ts
src/api/client.ts
src/api/types.ts
src/App.tsx
src/components/CompileResult.tsx
src/components/IngestPanel.test.tsx
src/components/IngestPanel.tsx
src/components/LanguageSwitcher.tsx
src/components/QueryBar.examples.test.ts
src/components/QueryBar.tsx
src/components/RallyGrid.tsx
src/components/SelectionFooter.tsx
src/hooks/useIngestJob.test.ts
src/hooks/useIngestJob.ts
src/hooks/useRallySelection.ts
src/i18n/hygiene.test.ts
src/i18n/i18n.test.tsx

```

```
src/i18n/index.ts
src/i18n/locales/en/common.json
src/i18n/locales/hi/common.json
src/i18n/locales/kn/common.json
src/index.css
src/lib/errors.test.ts
src/lib/errors.ts
src/lib/floor.test.ts
src/lib/floor.ts
src/lib/format.test.ts
src/lib/format.ts
src/lib/log.ts
src/lib/query.test.ts
src/lib/query.ts
src/lib/selection.test.ts
src/lib/selection.ts
src/main.tsx
src/test/setup.ts
```

10. assistant (2026-06-21T13:35:07.893Z)

The Glob path was being prefixed. Let me read the vite config, index.html, API client, and i18n setup.

11. user (2026-06-21T13:35:08.371Z)

```
1      import { defineConfig } from "vitest/config";
2      import react from "@vitejs/plugin-react";
3      import tailwindcss from "@tailwindcss/vite";
4
5      // The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
6      // proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
7      // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local
engine
8      // (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if
needed.
9      export default defineConfig({
10        plugins: [react(), tailwindcss()],
11        server: {
12          proxy: {
13            "/api": { target: "http://localhost:8000", changeOrigin: true },
14          },
15        },
16        test: {
17          // Default env is node (the pure lib/parser tests); the i18n render test opts into
jsdom via a
18          // `/// @vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
19          environment: "node",
20          include: ["src/**/*.test.{ts,tsx}"],
21          // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files
no-op.
22          setupFiles: ["src/test/setup.ts"],
23        },
24      });
25
```

12. user (2026-06-21T13:35:08.849Z)

```
1      <!doctype html>
2      <html lang="en">
3        <head>
4          <meta charset="UTF-8" />
5          <meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```

6      <title>KhelSutra Studio</title>
7    </head>
8    <body>
9      <div id="root"></div>
10     <script type="module" src="/src/main.tsx"></script>
11   </body>
12 </html>
13

```

13. user (2026-06-21T13:35:09.821Z)

```

1    // Typed client for the engine FastAPI. SAME-ORIGIN relative `/api/*` (the appliance
serves
2    // UI + engine behind one origin; in dev Vite proxies `/api` ? :8000). Every call
carries a
3    // correlation id (header + logs) so a journey is traceable end to end.
4    import { log, newCorrelationId } from "../lib/log";
5    import type { RallySegment, QueryResponse, CompileResponse, EngineConfig,
ProcessResponse, JobStatus } from "./types";
6
7    const BASE = "/api";
8
9    export class ApiError extends Error {
10      readonly cid?: string;
11      readonly status?: number;
12      readonly body?: string;
13      constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
14        super(message);
15        this.name = "ApiError";
16        this.cid = opts.cid;
17        this.status = opts.status;
18        this.body = opts.body;
19        if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
20      }
21    }
22
23    interface RequestOpts {
24      method?: string;
25      body?: string;
26      headers?: Record<string, string>;
27      /** Correlation id to thread through this call; auto-generated if absent. */
28      cid?: string;
29    }
30
31    async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
32      const cid = opts.cid ?? newCorrelationId();
33      const url = `${BASE}${path}`;
34      const method = opts.method ?? "GET";
35      log("info", "api.request", { cid, method, url });
36
37      let res: Response;
38      try {
39        res = await fetch(url, {
40          method,
41          body: opts.body,
42          headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {} ) },
43        });
44      } catch (err) {
45        // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
46        log("error", "api.network_error", { cid, method, url, error: String(err) });
47        throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
48      }
49

```

```

50     if (!res.ok) {
51         const body = await res.text().catch(() => "");
52         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
53         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
54     }
55
56     const data = (await res.json()) as T;
57     log("info", "api.response", { cid, method, url, status: res.status });
58     return data;
59 }
60
61 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
62 * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
63 * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
64 export async function processVideo(videoPath: string, cid?: string):
Promise<ProcessResponse> {
65     return request<ProcessResponse>("/process", {
66         method: "POST",
67         body: JSON.stringify({ video_path: videoPath }),
68         cid,
69     });
70 }
71
72 /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
73 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
74     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
75 }
76
77 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
78 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
79     const data = await request<QueryResponse>("/query", {
80         method: "POST",
81         body: JSON.stringify({ video_id: videoId }),
82         cid,
83     });
84     return data.play_segments ?? [];
85 }
86
87 /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
88 export async function compileReel(
89     videoId: string,
90     segmentIds: number[],
91     outputFilename: string,
92     cid?: string,
93 ): Promise<CompileResponse> {
94     return request<CompileResponse>("/compile", {
95         method: "POST",
96         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename }),
97         cid,
98     });
99 }
100
101 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
102 export function highlightUrl(videoId: string, filename: string): string {

```

```

103     return
104 }
105
106 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
107 export async function getConfig(cid?: string): Promise<EngineConfig> {
108     return request<EngineConfig>("/config", { method: "GET", cid });
109 }
110
111 /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
112 [main.py:362]. */
113 export function minShotCount(cfg: EngineConfig): number | undefined {
114     return cfg.stitching?.min_shot_count;
115 }

```

14. user (2026-06-21T13:35:09.838Z)

```

1     node_modules
2     dist
3     coverage
4     *.local
5     .vite
6

```

15. assistant (2026-06-21T13:35:12.607Z)

Now let me look at the i18n setup, the dist build output, the README, and the engine's backend/api static serving.

16. user (2026-06-21T13:35:13.575Z)

```

1     // i18n scaffold (tracker B1) ? the shared localization foundation for KhelSutra Studio.
2     //
3     // Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4     //   - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5     //   - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
6     the catalog,
7     //   not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
8     //   - one negotiation order shared with the marketing site (B3);
9     //   - always-fallback to `en` so a missing translation degrades to English, never a raw
10    key.
11    import i18n from "i18next";
12    import { initReactI18next } from "react-i18next";
13    import LanguageDetector from "i18next-browser-languagedetector";
14    import ICU from "i18next-icu";
15    import enCommon from "../locales/en/common.json";
16    import knCommon from "../locales/kn/common.json";
17    import hiCommon from "../locales/hi/common.json";
18
19    // Supported locales (D6): `en` + `kn` + `hi`. Anything else falls back to `en`.
20    export const SUPPORTED_LOCALES = ["en", "kn", "hi"] as const;
21    export type Locale = (typeof SUPPORTED_LOCALES)[number];
22    export const DEFAULT_LOCALE: Locale = "en";
23
24    // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
25    B3), so a
26    // visitor's language choice carries across both surfaces.
27    export const LANG_STORAGE_KEY = "khelesutra.lang";
28
29    // Catalogs are co-located here (D9). `kn` + `hi` shipped (B5/B6, AI-drafted ? native
30    review
31    // pending). A missing key in any locale still falls back to `en` ? the by-design
32    invariant.

```

```

28     const resources = {
29       en: { common: enCommon },
30       kn: { common: knCommon },
31       hi: { common: hiCommon },
32     } as const;
33
34     void i18n
35       .use(ICU)
36       .use(LanguageDetector)
37       .use(initReactI18next)
38       .init({
39         resources,
40         fallbackLng: DEFAULT_LOCALE,
41         supportedLngs: [...SUPPORTED_LOCALES],
42         nonExplicitSupportedLngs: true, // en-IN -> en
43         defaultNS: "common",
44         ns: ["common"],
45         interpolation: { escapeValue: false }, // React already escapes
46         returnNull: false,
47         // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
48         detection: {
49           order: ["localStorage", "querystring", "navigator"],
50           lookupLocalStorage: LANG_STORAGE_KEY,
51           lookupQuerystring: "lang",
52           caches: ["localStorage"],
53         },
54         react: { useSuspense: false },
55       });
56
57       // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
Reflects the
58       // resolved content language, so an `en` fallback for an absent locale still reads as
`en`.
59       function syncDocumentLang(lng: string) {
60         if (typeof document !== "undefined") {
61           document.documentElement.lang = (i18n.resolvedLanguage || lng ||
DEFAULT_LOCALE).split("-")[0];
62         }
63       }
64       i18n.on("languageChanged", syncDocumentLang);
65       syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);
66
67       export default i18n;
68

```

17. user (2026-06-21T13:35:14.060Z)

```

1     # web/ ? KhelSutra Studio (operator console SPA)
2
3     Vite + React + TS + Tailwind v4. The browser UI that **drives the rally indexer
engine**:
4     upload ? index ? pick rallies ? reel ? download. See `docs/PER_RALLY_SELECTION.md` (the
5     per-rally curation screen) and `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` (the single-box
appliance
6     this SPA fronts).
7
8     ## Dev
9     ```
10    npm install
11    npm run dev      # http://localhost:5173 ; proxies /api ? http://localhost:8000 (the
engine)
12    ```
13    Run the engine alongside it: `python -m backend.main --server` (in the badminton-
highlight-indexer
14    repo). The SPA uses **same-origin** relative `/api/*` calls ? in the appliance a reverse

```



```

proxy serves
15     this bundle and proxies `/api` to the engine on `127.0.0.1:8000`.
16
17     ## Checks (CI gates merge)
18     ```
19     npm run typecheck # tsc --noEmit
20     npm run test      # vitest run (unit tests; e.g. the typed API client)
21     npm run build     # vite build
22     ```
23
24     ## Layout
25     - `src/api/client.ts` ? typed engine client (`listRallies` / `compileReel` /
`highlightUrl` / `getConfig`),
26     grounded in the verified engine contract; every call carries a correlation id.
27     - `src/api/types.ts` ? engine data types (honest: `server`/`receiver`/`events` are
motion-only
28     fabrications and must not be surfaced).
29     - `src/lib/log.ts` ? structured client logging + correlation ids (observability seed,
appliance $5).
30     - `src/App.tsx` ? the console shell (screens land in M2+).
31

```

18. user (2026-06-21T13:35:15.687Z)

```

./assets/index-8AKPRqKx.css
./assets/index-BkCjbkEL.js
./assets/noto-sans-devanagari-devanagari-wght-normal-DWI_L5Cu.woff2
./assets/noto-sans-devanagari-latin-ext-wght-normal-CLKupaxV.woff2
./assets/noto-sans-devanagari-latin-wght-normal-Bxfr4UJH.woff2
./assets/noto-sans-kannada-kannada-wght-normal-DuWB-P8d.woff2
./assets/noto-sans-kannada-latin-ext-wght-normal-MdBBF41x.woff2
./assets/noto-sans-kannada-latin-wght-normal-DeLZCgFT.woff2
./index.html
---DIST-DU---
669K .

```

19. assistant (2026-06-21T13:35:19.318Z)

The build output is a standard Vite static bundle (~669K, single JS/CSS + woff2 fonts, all under `dist/`). The i18n catalogs are imported/bundled INTO the JS (not separate runtime files). Now let me check the engine side ? the static serving and the `backend/api/static` dead-code question.

20. user (2026-06-21T13:35:23.372Z)

```

error: Your local changes to the following files would be overwritten by merge:
    backend/main.py
Please commit your changes or stash them before you merge.
Updating 43eccc1..e68b583
Aborting
---STATIC DIR---
backend/api/auth.py
backend/api/job_worker.py
backend/api/static/app.js
backend/api/static/index.html
backend/api/static/style.css
backend/api/uploads.py
backend/api/__init__.py
backend/api/__pycache__/job_worker.cpython-313.pyc
backend/api/__pycache__/uploads.cpython-313.pyc
backend/api/__pycache__/__init__.cpython-313.pyc
---STATICFILES MENTIONS---
total 48
drwxr-xr-x 1 avidu 197609    0 Jun 11 20:44 .

```

```
drwxr-xr-x 1 avidu 197609      0 Jun 21 18:49 ..
-rw-r--r-- 1 avidu 197609 16350 Jun 11 20:44 app.js
-rw-r--r-- 1 avidu 197609  8743 Jun  7 14:14 index.html
-rw-r--r-- 1 avidu 197609 13418 Jun  7 14:14 style.css
```

21. user (2026-06-21T13:35:23.530Z)

```
backend\main.py:24:from fastapi.staticfiles import StaticFiles
backend\main.py:25:from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
backend\main.py:64:# Serves static frontend files directly (now under api/ per approved
structure)
backend\main.py:65:os.makedirs("backend/api/static", exist_ok=True)
backend\main.py:66:app.mount("/static", StaticFiles(directory="backend/api/static"),
name="static")
backend\main.py:70:    index_path = "backend/api/static/index.html"
backend\main.py:74:    return "<h3>Sports Highlight Indexer Server is Running. Please create
index.html in backend/api/static/</h3>"
backend\main.py:406:    return FileResponse(path, media_type=media, filename=name)
backend\api\static\index.html:7:    <link rel="stylesheet" href="/static/style.css">
backend\api\static\index.html:145:    <script src="/static/app.js"></script>
backend\worker\_main_.py:123:    timeout/failure, distinct from the local 2/3) rather than
letting a raw traceback escape.""
backend\infrastructure\database.py:63:    @staticmethod
backend\infrastructure\database.py:276:        # One model version per user (NULLs compare
distinct in SQLite, so the local
backend\infrastructure\database.py:284:        # (SQLite treats NULL user_id rows as distinct,
so the local install's single
backend\infrastructure\database.py:632:        # $0 indistinguishably from a free run (a new SKU
/ typo / gpu_type=None). Surface it loudly +
backend\config\models.py:34:    wsl_distro: str = "Ubuntu"
backend\config\models.py:61:    wsl_distro: str = "Ubuntu"
backend\eval\ablation.py:60:# Source: distill_local.FEATURE_NAMES (shuttle, includes
net_crossings), fusion_features
backend\eval\ablation.py:110:    the STATIC ``Variant(min_crossings=2)`` (no classifier); for
a learned ablation it carries
backend\eval\ablation.py:366:def static_gate_a_signal() -> Signal:
backend\eval\ablation.py:368:    set). Ablating it against the static
``Variant(min_crossings=2)`` full reproduces the
backend\eval\ablation.py:374:    ""The Gate-A reproduction config: full = STATIC
``Variant(min_crossings=2)`` (no learned
backend\eval\ablation.py:378:    return AblationConfig(full=full,
signals=[static_gate_a_signal()], granularity="signal")
backend\eval\ablation.py:408:    (distinct from the static Gate-A litmus).""
backend\eval\ablation.py:422:        f" . {'LOVO-honest' if report.honest_lovo else
'static'}"
backend\eval\ablation.py:462:        help="reproduce the static Gate-A measurement
(CONTROL vs min_crossings=2) "
backend\eval\ablation_pdf.py:68:        f"{'honest LOVO' if report.honest_lovo else 'static
re-score'}. "
backend\utils\geometry.py:11:def get_distance(p1: Tuple[float, float], p2: Tuple[float, float])
-> float:
backend\utils\geometry.py:12:    ""Calculate the Euclidean distance between two points (x,
y).""
backend\utils\geometry.py:20:    dist = get_distance(c1, c2)
backend\utils\geometry.py:21:    return dist * fps
backend\assets\fonts\README.md:5:only), so they can be redistributed with the tool and self-
hosted derivatives.
backend\assets\fonts\LICENSE-Apache-2.0.txt:6:    TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND
DISTRIBUTION
backend\assets\fonts\LICENSE-Apache-2.0.txt:11:        and distribution as defined by Sections 1
through 9 of this document.
backend\assets\fonts\LICENSE-Apache-2.0.txt:71:        publicly display, publicly perform,
sublicense, and distribute the
backend\assets\fonts\LICENSE-Apache-2.0.txt:90:    4. Redistribution. You may reproduce and
distribute copies of the
```

```

backend\assets\fonts\LICENSE-Apache-2.0.txt:102:         that You distribute, all copyright,
patent, trademark, and
backend\assets\fonts\LICENSE-Apache-2.0.txt:108:         distribution, then any Derivative
Works that You distribute must
backend\assets\fonts\LICENSE-Apache-2.0.txt:112:         of the following places: within a
NOTICE text file distributed
backend\assets\fonts\LICENSE-Apache-2.0.txt:119:         notices within Derivative Works that
You distribute, alongside
backend\assets\fonts\LICENSE-Apache-2.0.txt:126:         for use, reproduction, or distribution of
Your modifications, or
backend\assets\fonts\LICENSE-Apache-2.0.txt:128:         reproduction, and distribution of the Work
otherwise complies with
backend\assets\fonts\LICENSE-Apache-2.0.txt:151:         appropriateness of using or redistributing
the Work and assume any
backend\assets\fonts\LICENSE-Apache-2.0.txt:166: 9. Accepting Warranty or Additional
Liability. While redistributing
backend\assets\fonts\LICENSE-Apache-2.0.txt:199: distributed under the License is distributed
on an "AS IS" BASIS,
backend\utils\run_telemetry.py:207:     @staticmethod
backend\utils\run_telemetry.py:214:     @staticmethod
backend\eval\distill_local.py:1: """Distill Gemini (teacher) into a LEARNED local rally
classifier (student).
backend\eval\distill_local.py:21:     python -m backend.eval.distill_local --manifest videos.json
--model-out output/local_rally_model.json
backend\eval\distill_local.py:106:     print(f"=== Distilled local rally classifier vs Gemini
silver-GT ")
backend\eval\distill_local.py:152:     ap = argparse.ArgumentParser(description="Distill Gemini
labels into a local rally classifier.")
backend\pipeline\stitcher\compiler.py:27:     @staticmethod
backend\pipeline\stitcher\compiler.py:37:     @staticmethod
backend\pipeline\stitcher\compiler.py:50:     @staticmethod
backend\pipeline\stitcher\compiler.py:80:     @staticmethod
backend\pipeline\stitcher\compiler.py:89:         duplicates/overlaps while preserving genuinely
distinct (gapped) rallies.
backend\pipeline\validators\blur.py:32:         # Sample 15 frames evenly distributed across the
video
backend\pipeline\personalization\serving.py:22: static ``idx_cfg["fusion"]`` ? it does NOT call
anything here. Wiring this resolver into that
backend\pipeline\segmenters\base.py:29:         Use isinstance(result, Failure) (or
result.is_success()) to distinguish
backend\eval\classifier.py:32:     @staticmethod
backend\pipeline\detectors\base.py:34:     deliberately static and model-agnostic). If the new
detector needs different
backend\pipeline\detectors\base.py:43: 7. No change to eval/calibration code paths (they consume
the CSV + the static
backend\pipeline\detectors\base.py:96:     """Shared ``wsl -d <distro> bash -c 'source <conda_sh>
&& <cmd>`` invocation.
backend\pipeline\detectors\base.py:99:     inherit WSL plumbing. Requires ``self.cfg`` with
``distro``, ``conda_sh``
backend\pipeline\detectors\base.py:108:         argv = ["wsl", "-d", self.cfg.distro, "bash",
"-c", full]
backend\pipeline\detectors\base.py:188:         # model-agnostic TrackNetRunner staticmethods (same
CSV schema + windowing);
backend\reporting\harvest.py:33:     distinguishes a real probe from the absence of one (the
fallback footgun)."""
backend\pipeline\segmenters\trajectory_hybrid.py:89:     @staticmethod
backend\pipeline\segmenters\trajectory_hybrid.py:98:     @staticmethod
backend\pipeline\detectors\native_wasb_runner.py:10: * NO ``WslCommandMixin`` (no ``wsl -d
<distro> bash -c``), NO ``conda activate`` (the
backend\pipeline\detectors\native_wasb_runner.py:262:     parse_trajectory_csv =
staticmethod(TrackNetRunner.parse_trajectory_csv)
backend\pipeline\detectors\native_wasb_runner.py:263:     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
backend\eval\calibrate_local.py:1: """Distill Gemini (teacher) into the LOCAL-ONLY pipeline
(student) ? calibrate the

```

```

backend\eval\experiment.py:26:    aggregate deltas. Mirrors `distill_local`'s honest train-on-
others/predict-
backend\eval\experiment.py:186:    The velocity-windowing / distill corpus-growth path persists
a LEANER
backend\eval\experiment.py:365:    evaluator's own IoU matcher). The honest-LOVO training step
from `distill_local`. """
backend\eval\experiment.py:384:    - **static variant** (no learned filter): predict each
video directly ? no training,
backend\eval\experiment.py:420:    else:                                # static variant
or pinned model
backend\pipeline\detectors\fusion_features.py:193:# feature columns appended after the shuttle
features in distill_local.FEATURE_NAMES).
backend\pipeline\detectors\stub_runner.py:120:    parse_trajectory_csv =
staticmethod(TrackNetRunner.parse_trajectory_csv)
backend\pipeline\detectors\stub_runner.py:121:    trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
backend\eval\golden_fixtures.py:16:    parse_trajectory_csv ?
distill_local.build_candidates(proposal=pinned)
backend\eval\golden_fixtures.py:32:    the replay: `distill_local.build_candidates` (shared
code, intentionally not modified here)
backend\eval\golden_fixtures.py:62:from backend.eval import distill_local
backend\eval\golden_fixtures.py:88:#: returns just `(vt, mg, mwd)` and
`distill_local.build_candidates` (shared code, deliberately
backend\eval\golden_fixtures.py:93:#: The 5 fps/resolution-invariant shuttle feature columns
produced by distill_local. Re-stated
backend\eval\golden_fixtures.py:98:assert tuple(distill_local.FEATURE_NAMES) ==
SHUTTLE_FEATURES, (
backend\eval\golden_fixtures.py:99:    "distill_local.FEATURE_NAMES drifted from the pinned
shuttle feature set"
backend\eval\golden_fixtures.py:146:    end an active run so the next rally forms a distinct
window.
backend\eval\golden_fixtures.py:242:    """The committed synthetic scenarios. Deterministic;
chosen so each exercises a distinct
backend\eval\golden_fixtures.py:344:    intervals, feature_rows =
distill_local.build_candidates(
backend\eval\fusion_golden.py:3:Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5
person-cue features per shuttle
backend\eval\fusion_golden.py:28:from backend.eval import distill_local
backend\eval\fusion_golden.py:33:# The 5 fps/resolution-invariant shuttle features distill_local
already computes.
backend\eval\fusion_golden.py:34:SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
backend\eval\fusion_golden.py:65:    intervals, x_shuttle = distill_local.build_candidates(traj,
fps, fw)
backend\eval\eval_stats.py:11:    many *videos* it wins on (the distribution-free test the repo
already leans on:
backend\eval\eval_stats.py:74:    distribution-free "did B win on enough *videos*?" check.
backend\pipeline\detectors\wasb_runner.py:20:# parse_trajectory_csv /
trajectory_to_action_windows are @staticmethods on TrackNetRunner.
backend\pipeline\detectors\wasb_runner.py:33:    distro: str = "Ubuntu"
backend\pipeline\detectors\wasb_runner.py:53:        distro=w.get("wsl_distro", "Ubuntu"),
backend\pipeline\detectors\wasb_runner.py:198:    parse_trajectory_csv =
staticmethod(TrackNetRunner.parse_trajectory_csv)
backend\pipeline\detectors\wasb_runner.py:199:    trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
backend\pipeline\detectors\wasb_infer.py:311:    frame indistinguishable from the on-disk one to
everything downstream.
backend\eval\per_user_eval.py:12:    - `experiment.predict` ? variant ? held-out intervals
(static variants need no fit);
backend\eval\per_user_eval.py:47:    - **static variant** (no learned filter): None ? `predict`
scores directly, no leakage.
backend\pipeline\detectors\tracknet_runner.py:33:    distro: str = "Ubuntu"
backend\pipeline\detectors\tracknet_runner.py:49:        distro=tn.get("wsl_distro",
"Ubuntu"),
backend\pipeline\detectors\tracknet_runner.py:236:    @staticmethod
backend\pipeline\detectors\tracknet_runner.py:255:    @staticmethod

```

```

backend\pipeline\detectors\tracknet_runner.py:273:         dist = ((p.x - prev.x) **
2 + (p.y - prev.y) ** 2) ** 0.5
backend\pipeline\detectors\tracknet_runner.py:274:         velocity = (dist /
frame_width) * (fps / dframes)
backend\pipeline\detectors\tracknet_runner.py:280:         @staticmethod
backend\pipeline\detectors\tracknet_runner.py:401:         @staticmethod
backend\pipeline\detectors\tracknet_runner.py:430:         @staticmethod
backend\pipeline\detectors\tracknet_runner.py:438:         @staticmethod
backend\pipeline\detectors\tracknet_runner.py:478:         @staticmethod
backend\eval\regression.py:57:     # True only when a baseline existed to compare against.
Distinguishes a genuine PASS
backend\eval\rally_seg_eval.py:303:     0/0 recall would distort the mean. ``n`` per cell reports
the videos that survived that gate."""
backend\eval\served_gate_a.py:5:**permissive proposal windowing** (``distill_local.PROPOSAL`` =
recall-oriented) re-scored
backend\eval\segmentation_metrics.py:22:background segments strictly alternate, so the edit
distance collapses to (twice) the
backend\eval\splits.py:10: distill_local / run_regression deferred to later PR.
backend\eval\serve_contrast.py:50:     Builds candidates under ``preset``, then scores two static
variants vs ``gts``:
backend\eval\tolerance_metrics.py:6:boundary-error distributions + an over-seg guard
(split/merge counts) + a ?-sweep ? never one
backend\eval\tolerance_metrics.py:128:     ends fail the gate and vanish from the distribution),
which would HIDE the very end-deficit
backend\eval\tolerance_metrics.py:131:     def dist(vals: List[float]) -> Dict[str, float]:
backend\eval\tolerance_metrics.py:147:     return {"dstart": dist(ds), "dend": dist(de)}
backend\eval\tolerance_metrics.py:263:     """The full R2 block for one video: P/R/F1@? +
boundary-error distributions + over-seg guard."""
backend\eval>windowing.py:3:**The bug this closes.** The offline eval substrate
(``distill_local.build_candidates`` ?
backend\eval>windowing.py:25:caller opts in (``distill_local`` re-points its module-level
presets here, byte-identically).
backend\eval>windowing.py:82:     ``trajectory_to_action_windows`` / ``distill_local`` call
sites already speak. (The
backend\eval>windowing.py:137:#: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the
existing golden JSONs stay valid.

```

22. assistant (2026-06-21T13:35:27.988Z)

The engine HAS a live static mount at `/static` and serves `backend/api/static/index.html` at the root route. This is NOT dead ? it's an old vanilla-JS UI (app.js 16KB, no build step). Let me read the engine's main.py around the static-serving routes to understand exactly what's mounted and how, and confirm the relationship to the khelsutra SPA.

23. user (2026-06-21T13:35:29.167Z)

```

55     _cors_origins = ["*"]
56     app.add_middleware(
57         CORSMiddleware,
58         allow_origins=_cors_origins,
59         allow_credentials=False,
60         allow_methods=["*"],
61         allow_headers=["*"],
62     )
63
64     # Serves static frontend files directly (now under api/ per approved structure)
65     os.makedirs("backend/api/static", exist_ok=True)
66     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
67
68     @app.get("/", response_class=HTMLResponse)
69     def serve_index():
70         index_path = "backend/api/static/index.html"
71         if os.path.exists(index_path):
72             with open(index_path, "r", encoding="utf-8") as f:
73                 return f.read()

```

```

74         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
75
76     # Global variables initialized at startup
77     db_instance: Optional[Database] = None
78     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
79
80     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
81     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
82     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
83     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
84     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
85     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
86     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
87     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
88         JobWaiting,
89         _arm_wake_timer,
90         _drain_due_jobs,
91         _get_executor,
92         _job_to_request,
93         _MAX_DRAIN_WAKE_SEC,
94         _maybe_record_job_cost,
95         _run_claimed_job,
96         _schedule_next_drain_if_waiting,
97     )
98
99     # Legacy thin wrapper for any remaining direct calls during transition.
100    # New code should import load_app_config / AppConfig from backend.config directly.
101    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
102        cfg = load_app_config(config_path)
103        return cfg.model_dump(mode="json")
104
105    # --- API Models ---
106    class ProcessRequest(BaseModel):
107        video_path: str
108        player_pool: Optional[List[str]] = None
109        max_frames: Optional[int] = None
110        segmenter: Optional[str] = None
111
112    class QueryRequest(BaseModel):
113        video_id: str
114        server: Optional[str] = None
115        receiver: Optional[str] = None
116        duration_min: Optional[float] = None
117        event_type: Optional[str] = None
118
119    class CompileRequest(BaseModel):
120        video_id: str
121        segment_ids: List[int]
122        output_filename: str
123
124    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
125        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
126
127        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
128        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
129        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
130        """
131        if segments:
132            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)

```

```

133         else:
134             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
135
136
137     # --- API Endpoints ---
138     @app.get("/api/config")
139     def get_config():

```

24. assistant (2026-06-21T13:35:33.680Z)

Now let me check the engine's old static index.html to confirm it's the legacy vanilla UI, look at the khelsutra appliance architecture doc (the live design), and check the khelsutra docs index. Let me also check the local modification to backend/main.py that blocked the pull (the working-tree change).

25. user (2026-06-21T13:35:34.800Z)

```

1      <!DOCTYPE html>
2      <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <meta name="viewport" content="width=device-width, initial-scale=1.0">
6          <title>Badminton Highlight & Rally Indexer</title>
7          <link rel="stylesheet" href="/static/style.css">
8      </head>
9      <body>
10         <div class="app-container">
11             <!-- HEADER -->
12             <header>
13                 <div class="logo-section">
14                     <h1 id="app-title">Badminton Highlight & Rally Indexer</h1>
15                     <p>Local AI-powered video validation, segmentation, and clip
indexing</p>
16                 </div>
17                 <div class="status-badge">
18                     <span class="status-indicator"></span>
19                     <span>Local Server Active</span>
20                 </div>
21             </header>
22
23             <!-- MAIN GRID LAYOUT -->
24             <div class="main-grid">
25                 <!-- SIDEBAR: Controls & Validation -->
26                 <div class="sidebar">
27                     <!-- Video File Input Panel -->
28                     <div class="glass-panel" id="video-input-panel">
29                         <h2 class="panel-title">
30                             <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path
d="M23 19a2 2 0 0 1-2 2H3a2 2 0 0 1-2 2V8a2 2 0 0 1 2-2h4l2-3h6l2 3h4a2 2 0 0 1 2 2z"/><circle
cx="12" cy="13" r="4"/></svg>

```

26. user (2026-06-21T13:35:36.910Z)

```

backend/main.py | 29 ++++++-----
1 file changed, 22 insertions(+), 7 deletions(-)
---WHAT CHANGED IN main.py (static-relevant?)---

```

27. user (2026-06-21T13:35:37.371Z)

No files found

28. assistant (2026-06-21T13:35:41.891Z)

The uncommitted main.py change does NOT touch static serving (it's the M9 auth work on this branch). So the engine currently serves a legacy vanilla-JS UI at `/` and `/static`, completely separate from the khelsutra Vite SPA. Let me read the live appliance architecture doc and the khelsutra docs.

29. user (2026-06-21T13:35:43.554Z)

```
docs/ANNOTATE_AND_TRAIN.md
docs/DRY_RUN_CUJ.md
docs/FRIENDLY_AND_MULTILINGUAL.md
docs/LOCAL_APPLIANCE_ARCHITECTURE.md
docs/PER_RALLY_SELECTION.md
docs/PORTABILITY.md
docs/README.md
docs/REQUEST_ACCESS_FORM.md
---SITE---
README.md
access.db
make_og.py
node_modules
package-lock.json
package.json
public
schema.sql
scripts
server.js
src
wrangler.toml
---SITE-SCRIPTS---
droplet-smoke.sh
parity-check.sh
provision-vm.sh
teardown-vm.sh
```

30. user (2026-06-21T13:35:46.345Z)

```
1      # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3      > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20
4      > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** $9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6      > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7      > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9      ---
10
11     ## 0. TL;DR
12     **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
degenerate collapse of the same storage/worker split.
13
14     The **single most important caveat:** the engine has **zero app-layer auth and CORS
`***` `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
```



```

bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
and a reverse proxy/tunnel is the sole authenticated ingress ? the gate, not the app, is the
tenancy boundary.
15
16     Two honest hardware truths: (1) the current droplet is CPU-only ? real detection
there is Gemini (cloud) or stub (CPU mock); real local-GPU detection needs a GPU
host (native WASB, in-flight). (2) The owner priority is to stand up a GPU host ASAP ?
provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs in parallel
with the UI MVP, which ships on Gemini today.
17
18     ## 1. Problem & goal
19     The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
{infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the
FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified
backend/main.py]`. What's missing is a console that lets a non-CLI operator run the whole
loop on a single box, honestly, behind a gate.
20
21     Headline CUJ (owner's words): a hosted website where I pick an input ? a
Google Drive file, a bucket file (`s3://`/`gs://`), or a local upload ? it lands in the
bucket (DO Spaces / GCS) or locally ? I generate highlights on a GPU-enabled host
running the real e2e ? I pick rallies, watch, and download ? completely from the UI.
22
23     Outcome ("good"): an operator signs in, picks a source, watches it index, curates
rallies, downloads a reel ? on the live CPU droplet today (Gemini/stub), and on a GPU host
later (native WASB) with no UI fork.
24
25     Read to get current: `PER_RALLY_SELECTION.md` ($3/$4/$10 contract anchors), engine
`HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
`VIDEO_LOCALITY_MODEL.md` (privacy/retention).
26
27     ## 2. Decisions locked
28     | # | Decision | Source / date | Implication |
29     |---|---|---|---|
30     | D1 | Reverse-proxied single origin; engine stays `127.0.0.1:8000` | `[verified
main.py:638]`; 2026-06-20 | CORS `` is moot from the browser ? the proxy is the boundary |
31     | D2 | Edge auth, zero engine auth code | `HOSTING_PLAN.md`; 2026-06-20 | Caddy
basicauth OR cloudflared + Cloudflare Access |
32     | D3 | MVP drives the in-process FastAPI worker loop, `storage=local` | `[verified
job_worker.py:37, local.py:33]`; 2026-06-20 | bucket worker = reserved scale path, out of MVP |
33     | D4 | Embed RallyPicker from `PER_RALLY_SELECTION.md` verbatim | PR [#11];
2026-06-20 | the console is a superset; do not redesign the picker |
34     | D5 | Console lives in the khelsutra web React scaffold | `web/README.md`;
2026-06-20 | Cloudflare-Pages-style root, not engine static |
35     | D6 | Compute placement is a setting, rendered from `GET /api/capabilities` |
`[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |
36     | D7 | Ingest is a source menu (local upload · bucket URI · Drive) normalizing to
one video object | `[verified backend/storage/ref.py parse_uri]`; 2026-06-20 | Drive is a
stub today ? scoped honestly ($4.2) |
37     | D8 | GPU host is provider-agnostic (DO droplet / GCE VM / Cloud Run+GPU); storage
stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-
in |
38     | D9 | Observability is first-class ? structured logs on every new path + a DoD item
| Owner 2026-06-20 | see $5; verified during CUJ replay |
39     | D10 | Delivery = a self-hosted webserver + UI the user spins up (NOT a desktop
app); the user points at local storage + GPU/CPU OR cloud locations and the UI drives the
tool | Owner 2026-06-20 | supersedes `DESKTOP_APP_PLAN.md`; one artifact (this appliance)
runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41     ## 3. Foundation ? what the engine gives us today `[verified this session unless
noted]`
42     - FastAPI loop: `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ?
`POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}`
(FileResponse) `[main.py]`. Chunked upload exists ? `POST /api/upload/init` · `PUT
/api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}`
`[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` never calls them (it only

```

hits `/api/process` + `/api/compile` `[verified app.js:88,373]`. So browser upload is **client-wiring only**, no engine change.

43 - **In-process worker**: one module-global `ThreadPoolExecutor(max\_workers=1)`
[verified job_worker.py:37] ? strictly **one job at a time**; the DB `jobs` table is the
clock; `fail\_stale\_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a
separate knob `indexing.ai\_max\_workers` (**default 5** [verified models.py:183]; set to 1
to serialize) ? it is **not** already 1.

44 - **Storage**: `storage.backend` default = "local" [verified models.py:298];
`LocalStorage.fetch\_to\_local` is **identity** [verified local.py:33], `put` no-ops on
`src==dst` ? so on a single box the worker's pull?pipeline?push **collapses** to read-local/write-
local. URI scheme `parse\_uri` [verified ref.py]: bare `local://` ? local; `s3://`
(AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub**
(emulated only). Credentials never in config ? boto3 default chain / `~/aws` [verified
s3.py].

45 - **Compute seam**: `indexing.detector\_impl` ? {`auto`=WSL, `native`=Linux-native,
`stub`=CPU mock}; default "auto" [verified models.py:129], resolved in
`trajectory\_hybrid.\_resolve\_runner` [verified :76-85]. **Native WASB** is in-flight on branch
`feat/native-wasb-runner`: **native_wasb_runner.py** **exists** [verified];
fusion_hybrid._build_native_runner returns `NativeWasbRunner` [verified
fusion_hybrid.py:86] while **trajectory_hybrid._build_native_runner** still raises [verified
:57-63]. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ?
not exercisable on the droplet or in the offline suite. Treat native as **not production-**
verified.

46 - **Worker CLI**: `python -m backend.worker {infer|train}` pulls a `--video-uri` ?
pipeline ? pushes `outputs/<video\_id>/{intervals.csv,report.json}` [verified
worker/__main__.py:88,149]. The validated GPU-free/key-free combo: `detector\_impl=stub` +
`--set indexing.skip\_ai\_handoff=true` + `storage.s3`.

47 - **Gating**: cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` [HOSTING_PLAN.md]. `GET
/api/health` **now EXISTS** (engine M1, [#268]) ? {`status`, `sport`, `default_segmenter`}
[verified main.py:127, corrected 2026-06-21 ? was "does not exist"].

48 - **Droplet reality**: `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts
the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the
full train+infer e2e against DO Spaces with the **GPU mocked** [SETUP_NEW_MACHINE.md,
DECOUPLED_COMPUTE/HANDOFF.md]. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run
there.

49 - **Absent (net-new)**: `GET /api/capabilities`, `GET /api/videos` (only single-row
`get\_video(id)`), `GET /api/reels/{video\_id}`, any source/proxy streaming endpoint, any
Dockerfile, any per-video **cost ledger** [verified ? grep returns none].

50

51 ## 4. Design / architecture

52

53 ### 4.1 Topology & request path

54 ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy** /
static host serving the `web/` SPA bundle and proxying `/api/\*` ? engine **FastAPI**
`127.0.0.1:8000` ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
site stays a **separate** systemd unit on a **separate** subdomain.

55

56 ...

57 PUBLIC INTERNET

58 ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app

59 ? cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 +
basicauth

60 ?

61 ???

62 ? ONE BOX firewall BLOCKS inbound 8000 + 9000/9001/9023 ?

63 ? (2) reverse proxy / static host (Caddy/cloudflared) ?

64 ? /, /assets/* ? web/ React SPA /api/*, /api/highlights/* ? :8000 ?

65 ? ? ONE ORIGIN to the browser ? engine CORS '*' is irrelevant ?

66 ? ? ?

67 ? (3) ENGINE FastAPI (host=127.0.0.1:8000, HARD-bound [main.py:638]) ?

68 ? control: jobs table + ThreadPoolExecutor(max_workers=1) drain ?

69 ? data: uploads.py chunked upload · /api/compile · /api/highlights ?

70 ? ? ?

```

71      ? (4) STORAGE = LocalStorage (default)  fetch=identity · put=no-op(src==dst) ?
72      ?                                     ?
73      ? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet]          ?
74      ?                                     native WASB (GPU host) · auto (owner Win+WSL) ?
75      ?????????????????????????????????????????????????????????????????????
76      RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77      ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3:///` ?
outputs/<vid>/ back
78      (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
79      ...
80
81      ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82      All sources normalize to ***a video the pipeline can read*** ? a local path (MVP) or a
`StorageRef` (scale):
83
84      | Source | Path today | Status |
85      |---|---|---|
86      | **Local upload** | SPA chunks the file ? `max_part_mb` via the existing
`/api/upload/init\|part\|complete` `[verified uploads.py]` ? server path ? `POST /api/process`.
(Scale: also `StorageBackend.put` to `s3:///`/`gs:///`.) | **wire client only** (endpoints exist)
|
87      | **Bucket URI** (`s3:///`, `gs:///`) | **POST /api/process` accepts the bucket URI
directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283
? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the
**engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs`
is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket
URI 400.)* | **engine: DONE (verified); SPA ingest call: design (= B-P2)** |
88      | **Google Drive** (`gdrive:///`) | `parse_uri` recognizes it, but the gdrive backend is
an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]` | **stub ? roadmap;** MVP
requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90      ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91      | Placement | Runs | Needs | Real/mock | Honest caveat |
92      |---|---|---|---|---|
93      | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker |
`GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google
? **must be in consent copy**; **no per-video cost ceiling** today |
94      | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
demo/regression only |
95      | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-
key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96      | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ?
`NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud
Run+GPU**; `WASB_WEIGHTS` | **REAL**, on-device | partially wired (fusion builds it, trajectory
refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no
Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97      | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
| REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98      | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU
host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration
endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as
a setting" is **design** ? selector shown **disabled** in MVP |
99
100     **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit
for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the
**containerization feasibility check** above.
101
102     ### 4.4 Auth / exposure (non-negotiable invariant)
103     No engine auth, by design; the gate is external. **Invariant (all three must hold):**
engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist
`[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**;
set `security.allowed_video_root`; rate-limit; per-user quota.

```

```

104
105     ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106     - **S1 Ingest & Process** ? the source menu ($4.2) + a segmenter picker from
    `/api/capabilities` ? `POST /api/process`.
107     - **S2 Progress** ? 3s poll of the free-text stage
    (`hashing/validating/segmenting/finalizing`), a one job at a time notice, and the restart-
    failure terminal state.
108     - **S3 Rally pick ? reel** ? embed RallyPicker verbatim (`PER_RALLY_SELECTION.md`):
    honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter.
    Per-rally is ONE screen of the console.
109     - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for
    reload-survival.
110     - **S5 Settings** ? render `config` + `capabilities`; the compute/storage target
    selector shown DISABLED ("single-box: local") until the bucket-worker bridge lands.
111
112     ### 4.6 Reuse map
113     Reused (no re-plumb): the full `/api` loop; the in-process worker; the chunked
    upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing
    `NativeWasbRunner`; the RallyPicker design + anchors; the `web/` scaffold; the
    `HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. Net-new: Caddyfile + systemd units;
    `GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the
    SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker
    bridge.
114
115     ## 5. Observability ? logging & monitoring `[required, first-class]`
116     Launch-readiness directive (owner 2026-06-20): logs and monitoring taken seriously,
    with a full log-analysis pass across every new code path. The engine already uses a
    centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new
    path.
117
118     - Correlation ID end-to-end. Thread the existing `job_id` (from `/api/process`)
    plus an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and
    into the worker for the bucket path, so one CUJ is traceable across UI ? FastAPI ? worker ?
    GPU host. Log it on every line.
119     - Structured logs. New endpoints/screens emit structured (JSON-able) logs at entry
    / result / error ? not just "it ran". The SPA logs client-side errors + failed fetches with
    the same correlation id.
120     - Loud failures, never silent drops. Carry the engine's ethos (#240 "surface failed
    chunks loudly") + RallyPicker's `min_shot_count`-warning rule: a partial/empty reel must
    never look complete; every dropped clip/chunk is logged + surfaced in the UI.
121     - Health + metrics. Ship the missing `GET /api/health` `[HOSTING_PLAN.md:42]` and a
    thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a
    ledger exists) for the engine-mode banner + alerting.
122     - Aggregation for transient hosts. A scale-to-zero Cloud Run / GPU host is
    ephemeral ? logs must ship to a durable sink (the bucket or a log service), not die with the
    instance.
123     - Log-analysis pass before merge (DoD item). Every PR that adds a code path includes
    a check that the path logs meaningfully, and the logs are inspected during CUJ live-replay
    ($8 Layers 173). Ties [[feedback-launch-quality-bar]].
124
125     ## 6. Risk table
126     | Risk | Today | Mitigation |
127     |---|---|---|
128     | No app auth + CORS `` `[main.py:42]` | open if any of {localhost-bind, firewall,
    gate} slips | enforce all three; LIVE posture test that raw `:8000` is refused from outside
    |
129     | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes
    them |
130     | Retention | `output/`+`uploads/` originals never deleted | sweep / keep-proxy-only
    before any non-owner exposure |
131     | Surprise Gemini/GPU bill | no cost ceiling/ledger | per-video cap;
    `ai_max_workers` (default 5) tunable; `max_workers=1` job serialization |
132     | Native WASB unverified | partially wired; owner-parity-gated | treat as not-
    production-verified until owner byte-parity on a real GPU box |
133     | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist |

```

```

coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD,
explicit `git add`** |
134 | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require
Drive?bucket pre-step until real |
135 | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before
promising Cloud Run/GCE |
136 | Shared-checkout collisions | active worker sessions share the engine tree | the
branch-safety protocol on every engine-side PR |
137 | **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted
in chat or committed (cf. the **already-rotated Spaces key** exposed in chat,
`DECOUPLED_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive
/ `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token,
revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action
every time** |
138
139 ## 7. Honest limits ? what this does NOT do
140 Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs
global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make
the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source
playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not**
containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-
parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the
**console plumbing**, not engine rally-detection quality (a separate track).
141
142 ## 8. CUJs & live-test plan `[required]`
143 **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core);
**CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters +
honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144
145 **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146 - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing
the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure;
`query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ?
CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`).
The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start
dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel
link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug
(CUJ-3), capability honesty (CUJ-4).
147 - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main
--server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock
and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema,
compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock
handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148 - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1
with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety
posture test** (raw `:8000/`:900x` refused from outside) as an assertion, not an assumption;
reuse the `parity.yml` dual-host pattern.
149 - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job +
headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no
stacking**; coverage at/above the floor; **logs inspected during replay** ($5).
150 - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU
box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/`cv2`-vs-`ffmpeg`
behavior; a multi-GB browser upload. Reported back into $9, stated plainly as **not verified**
in the offline suite.
151
152 ## 9. Deliverables & progress tracker ? **source of truth**
153 Legend: ? Todo . ? In progress . ? Done . ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR
ships its tests + meaningful logs ($5).
154
155 | ID | Deliverable | Depends on | Gated? | Status | PR |
156 |----|-----|-----|-----|-----|----|
157 | T0 | This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers | ? | owner | ? |
(this PR) |
158 | T1 | Mock engine + capability fixtures + browser-replay CUJ scripts (L1) | T0 | no | ?

```

```

| ? |
159 | T2 | Appliance shell: Caddyfile + systemd units; firewall; **edge auth**; rotate key
(P1) | ? | owner | ? | ? |
160 | T3 | L3 live posture test (raw `:8000`/`:900x` refused from outside) | T2 | no | ? | ? |
|
161 | T4 | S1 ingest (local upload wired) + segmenter picker | T1 | no | ? | ? |
162 | T5 | S2 progress (poll + one-job + restart-failure) | T4 | no | ? | ? |
163 | T6 | S3 embed **RallyPicker** + compile + download | T4 | no | ? | ? |
164 | T7 | *(engine)* `GET /api/capabilities` + `GET /api/health` + banner | T1 | coord | ?
| ? |
165 | T8 | *(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival) |
? | coord | ? | ? |
166 | T9 | S5 Settings (config+capabilities; disabled compute/storage target) | T7,T8 | no |
? | ? |
167 | T10 | Observability pass: correlation id end-to-end + structured logs + metrics (§5) |
T4 | no | ? | ? |
168 | T11 | Retention sweep for `output/+`uploads/` | ? | owner | ? | ? |
169 | T12 | Ingest from **bucket URI** (`s3://`/`gs://`) wiring ? **engine half DONE**
(`/api/process` accepts URIs, M2/[#280], `[verified]`); remaining = the SPA ingest call (=
**B-P2**) | T4 | coord | ? | engine done; SPA pending |
170 | T13 | **GPU host stand-up** (provider-agnostic) + wire `detector_impl=native`; owner
byte-parity (P4) | branch | ? owner | ? | ? |
171 | T14 | *(reserved)* Bucket-worker bridge ? remote-GPU "compute as a setting" (P5) |
T8,T12 | ? owner | ? | ? |
172
173 **Owner priority (2026-06-20): T13 (GPU host) is elevated to run in parallel from the
start** ? owner provisions the host + token; the engine session lands native WASB; the UI MVP
does **not** block on it (ships on Gemini).
174
175 ## 10. Open questions ? owner / external
176 **Blocking gates:** (1) **edge-auth** ? Caddy basicauth (self-contained) vs
cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) **retention
defaults** ? auto-delete originals after compile? keep proxies+intervals only? window?
(privacy/consent). (3) **per-video Gemini/GPU cost ceiling** ? what cap, hard-fail vs warn? (4)
**GPU host choice** ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker
worth building for Cloud Run)? (5) **P5 bridge ownership** ? who builds the orchestration
endpoint + how it coordinates with the active worker sessions.
177 **Nice-to-knows:** console home (web/ vs engine static); a review-playback streaming
endpoint; per-user scoping timing; containerization vs runbook.
178
179 ## 11. Definition of done
180 - [ ] CUJ-1 works end to end on the droplet (Gemini **and** stub) behind the gate, via
the ingest menu.
181 - [ ] Raw `:8000`/`:900x` unreachable from outside (**tested**, not assumed).
182 - [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped
honestly (not faked).
183 - [ ] RallyPicker embedded with honesty rules; `min_shot_count` surfaced.
184 - [ ] `/api/capabilities` + `/api/health` + banner truthful; reload-survival via
`/api/videos` + `/api/reels`.
185 - [ ] **Observability:** correlation id threaded end-to-end; structured logs at every
new path; loud failures; logs inspected during CUJ replay.
186 - [ ] Retention sweep in place; `GEMINI_API_KEY` rotated; `allowed_video_root` set.
187 - [ ] CI green (L2 contract + L4 regression); coverage ? floor; §9 tracker all
??-resolved.
188
189 ## 12. References
190 **Engine code `[verified this session]`:**
`backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
`backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`;
`backend/api/static/app.js:88,344,373`;
`backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`;
`backend/config/models.py:129,183,298`;
`backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py,
stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter:

```

```
gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`;
`DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`;
`DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`;
`PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`,
`PORTABILITY.md`, `web/README.md`.
```

191

192 ---

193 ### Changelog

194 - 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs (FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor re-verified against source before writing (catching the native-WASB partial-wiring + `ai\_max\_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local), multi-provider GPU, and first-class Observability directives.

195

31. assistant (2026-06-21T13:35:51.567Z)

This is the authoritative design. Key insight (D1, D5): the appliance design assumes a reverse proxy serves the SPA and proxies `/api` ? NOT FastAPI serving the SPA. The console "lives in khelsutra web/, NOT engine static." Let me check the PORTABILITY doc (it likely discusses bundling/distribution) and the FRIENDLY_AND_MULTILINGUAL doc, plus confirm how the SPA references `/api` (verify there are no hardcoded absolute URLs beyond the BASE constant).

32. user (2026-06-21T13:35:52.521Z)

```
1      # Portability ? Cloudflare now, always-on VM if needed
2
3      > **Status:** design + proven (2026-06-20). **Decision:** keep the site/form host-
portable so
4      > moving off Cloudflare to an owner-run always-on VM ("if Cloudflare acts out") is an
**adapter
5      > swap, not a rewrite.**
6
7      ## The shape that makes it portable
8      - **Logic is runtime-agnostic.** `src/index.js` uses Web-standard `Request`/`Response`
and pure
9      helpers (`validate`, `toRow`, `isBot`, `handleRequest`) ? no host-specific calls.
10     - **All DB coupling is in adapters** (`src/store.js`): `d1Store` (Cloudflare D1) and
`sqliteStore`
11     (node:sqlite / better-sqlite3). `handleRequest(request, store)` takes the store by
injection.
12     - **One schema, one SQL.** `schema.sql` and the `INSERT` are plain SQLite, so D1 and a
VM's SQLite
13     file are byte-for-byte compatible ? zero dialect change. (Postgres later = minor
placeholder tweaks.)
14
15     ## Two hosts, same code
16     | | Cloudflare (now) | Always-on VM (fallback) |
17     |---|---|---|
18     | Entry | `src/index.js` (Worker `fetch`) | `server.js` (node:http) |
19     | Static | `env.ASSETS` | `server.js` serves `public/` |
20     | Storage | `d1Store(env.DB)` ? D1 | `sqliteStore(db)` ? `node:sqlite` file |
21     | Country | `cf-ipcountry` header | optional (GeoIP / null) |
22     | Deploy | git ? Workers Builds | `node server.js` under systemd |
23     | TLS/DNS | Cloudflare | keep Cloudflare in front, **or** Caddy/Let's Encrypt |
24
25     **Enforced, not asserted.** `node --test src/` runs BOTH paths and CI
(`.github/workflows/test.yml`,
26     Node 24) runs them on every PR, so the repo stays compatible with both hosts:
27     - `src/index.test.js` ? the Cloudflare Worker + D1 logic.
28     - `src/server.test.js` ? **boots the real `server.js`** (node:http + node:sqlite) and
round-trips it
29     over HTTP into SQLite (the same proof we run on the droplet, automated).
```

30

```

31     For a live check against a real VM: `SSH_TARGET=root@host scripts/droplet-smoke.sh` (scp
+ boot +
32     round-trip + cleanup). Verified live on the owner's droplet (Ubuntu 24.04, isolated
`/srv/khelsutra-site`).
33
34     ## Provider-portable provisioning ? destroy ? resume anywhere (scripted + TESTED)
35     Spinning up a fresh VM on any provider (DigitalOcean / GCE / any Ubuntu box) and
tearing it down
36     without losing data is a first-class, scripted flow ? so destroying a box and
resuming elsewhere is
37     routine, not risky.
38
39     - scripts/provision-vm.sh ? run from your LOCAL machine against a FRESH Ubuntu VM.
Pushes `site/`
40     via `git archive` (private-repo-safe ? no clone, token, or secret on the VM),
installs Node (?24,
41     so `node:sqlite` is unflagged), installs + starts a systemd service, opens the
firewall, and
42     verifies it serves (form + a real POST round-trip). Idempotent.
43     ...
44     SSH_TARGET=root@<ip> [SSH_KEY=~/.ssh/khelsutra_droplet] [PORT=8787]
[RESTORE_DB=./backup.db] \
45     site/scripts/provision-vm.sh
46     ...
47     - scripts/teardown-vm.sh ? exports the SQLite state DB to a local timestamped
backup BEFORE
48     anything is destroyed (registrations survive the move), stops the service, and
optionally destroys the
49     DO droplet (`DESTROY=1 DROPLET_ID=<id>`, via `doctl`).
50     ...
51     SSH_TARGET=root@<ip> [BACKUP_DIR=./vm-state-backups] [DESTROY=1 DROPLET_ID=<id>] \
52     site/scripts/teardown-vm.sh
53     ...
54     - Clean resume on a different provider: teardown-vm.sh (export) ? destroy ?
`RESTORE_DB=<backup>`
55     SSH_TARGET=root@<newip> site/scripts/provision-vm.sh`. The restored DB is `scp`d in
before the service starts.
56
57     Verified live (2026-06-20): a fresh DO Ubuntu-24.04 droplet went bare ? operational
+ publicly
58     serving (HTTP 200) via `provision-vm.sh` (Node 24 auto-installed, systemd unit,
firewall); `teardown-vm.sh`
59     exported the state DB (12 KB); a re-provision with `RESTORE_DB` preserved the
registration (`rows:2` =
60     restored + the verify POST) ? then the throwaway droplet was destroyed. The
destroy?resume cycle is
61     reproducible across providers.
62
63     ## VM deploy runbook ("if Cloudflare acts out")
64     Node 22+ only ? no `npm install` (node:sqlite is built in).
65     ...
66     # on the VM:
67     git clone ? && cd khelsutra/site          # or scp the site/ folder
68     PORT=8080 DB_PATH=/var/lib/khelsutra/access.db node server.js
69     # put it under systemd (auto-restart) + a reverse proxy (Caddy/nginx) for TLS,
70     # then point DNS at the VM. ~1 hour; logic + schema unchanged.
71     ...
72
73     ## Honest notes
74     - "Cloudflare acts out" usually means Workers/D1 pricing or limits, not DNS ? so the
cheapest
75     move is to keep Cloudflare DNS/CDN in front and shift only the compute to the VM
(point a
76     route/origin at the box). Full exit (own DNS + Caddy TLS) is also supported.
77     - You can run both (Cloudflare primary + VM warm standby) since the schema is

```



```

identical; a
78     periodic `access_requests` export keeps them in sync.
79     - `server.js` is intentionally dependency-free; if you later want Postgres or a
framework (Hono runs
80     on Workers *and* Node), it slots in behind the same `store` + handler.
81

```

33. user (2026-06-21T13:35:54.456Z)

```

C:\Users\avidu\Projects\khelsutra\web\src\api\client.ts:7:const BASE = "/api";
C:\Users\avidu\Projects\khelsutra\web\src\api\client.ts:33:  const url = `${BASE}${path}`;
C:\Users\avidu\Projects\khelsutra\web\src\api\client.ts:39:    res = await fetch(url, {
C:\Users\avidu\Projects\khelsutra\web\src\api\client.ts:103:  return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;

```

34. assistant (2026-06-21T13:36:00.959Z)

This confirms the critical fact: the SPA hardcodes `BASE = "/api"` as a SAME-ORIGIN relative path ? NO absolute URL, no env-var indirection, no `VITE\_ENGINE\_ORIGIN` actually consumed in code (the vite comment mentions it as a dev-proxy override but the client never reads it). So bundling the SPA behind the engine on one origin works with zero code change.

The PORTABILITY.md is about the marketing `site/`, not the `web/` SPA ? but it shows the pattern (`server.js` serving `public/`). Note `web/dist` is gitignored ? the build output is NOT checked in. Let me confirm how the `site/` is structured (it's a separate Cloudflare/node app) and check the engine's pyproject for whether any package-data/static inclusion exists, plus the FRIENDLY_AND_MULTILINGUAL doc briefly for i18n shipping.

35. user (2026-06-21T13:36:01.419Z)

```

1      # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2      #
3      # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4      # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5      # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6      # auto-detection would otherwise miss the un-__init__'d `backend` root.
7      #
8      # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9      # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10     # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11     # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12     [build-system]
13     requires = ["hatchling"]
14     build-backend = "hatchling.build"
15
16     [project]
17     name = "badminton-highlight-indexer"
18     version = "0.1.0"
19     description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20     readme = "README.md"
21     requires-python = ">=3.10"
22     license = { text = "MIT" }
23     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27     dependencies = [
28         "fastapi>=0.111.0",
29         "uvicorn>=0.30.1",
30         "opencv-python>=4.9.0.80",
31         "numpy>=1.26.4",
32         "pydantic>=2.7.1",
33         "jinja2>=3.1.4",

```

```

34     "python-dotenv>=1.0.0",
35     "google-genai>=2.4.0",
36     "supervision>=0.21.0,<0.30.0",
37 ]
38
39 [project.optional-dependencies]
40 # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41 dev = [
42     "pytest",
43     "pytest-cov",
44     "httpx",          # fastapi TestClient transport
45     "ruff",
46     "mypy",
47     "PyJWT[crypto]>=2.8", # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
48 ]
49
50 # M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
security.edge_auth_enabled=True.
51 # `jwt` is lazy-imported, so the base runtime works without this; install it for
hosted/multi-tenant.
52 auth = ["PyJWT[crypto]>=2.8"]
53
54 # Per-video observability reports (SOFT dependency: the pipeline degrades to a
55 # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
56 # fetched over SSH; if you lack access, simply omit this group ? reporting
57 # tests skip via pytest.importorskip. For local dev against a sibling checkout:
58 #   pip install -e ../sports-obsreport --no-build-isolation
59 reporting = [
60     "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
61 ]
62
63 # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
64 # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
65 # express, so they are NOT declarable here. Install them out-of-band instead:
66 #   # CUDA 12.4 box:
67 #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
68 #   # CPU-only box (also what CI uses):
69 #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
70 gpu = []
71
72 # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
73 # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
74 # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
75 # DO Spaces + MinIO via endpoint_url.) `gdrive` needs NO extra ? it reads a locally
76 # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
77 storage-s3 = ["boto3>=1.34"]
78 storage-gcs = ["google-cloud-storage>=2.16"]
79
80 [project.scripts]
81 # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
82 indexer = "backend.main:main"
83
84 [tool.hatch.metadata]
85 # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
86 # reference*); hatchling requires opting in to allow those in metadata.
87 allow-direct-references = true
88
89 [tool.hatch.build.targets.wheel]
90 # `backend` is a namespace package (no __init__.py); list both import roots
91 # explicitly so an editable/wheel install exposes `import backend` and
92 # `import training` regardless of cwd.
93 packages = ["backend", "training"]

```

36. user (2026-06-21T13:36:03.419Z)

```

import { useEffect, useMemo, useState } from "react";
import { useTranslation, Trans } from "react-i18next";
import { useRallySelection } from "../hooks/useRallySelection";
import { RallyGrid } from "../components/RallyGrid";
import { QueryBar } from "../components/QueryBar";
import { SelectionFooter } from "../components/SelectionFooter";
import { LanguageSwitcher } from "../components/LanguageSwitcher";
import { CompileResult, type BuildState } from "../components/CompileResult";
import { IngestPanel } from "../components/IngestPanel";
import { runQuery, type ParsedQuery } from "../lib/query";
import { floorRisk } from "../lib/floor";
import { reelFilename } from "../lib/format";
import { parseDetail } from "../lib/errors";
import { log, newCorrelationId } from "../lib/log";
import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount } from
"./api/client";
import type { EngineConfig, RallySegment } from "../api/types";

/** One compact, structured description of how a query was understood (for the trace log). */
function parseTrace(q: ParsedQuery) {
  return {
    raw: q.raw,
    recognized: q.recognized,
    filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
    sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
    limit: q.limit ?? null,
    unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
  };
}

// Demo data shown when no ?video=<id> is supplied. Shape matches the verified play_segments
// contract; only honest fields are surfaced. A real session loads from POST /api/query.
const DEMO_RALLIES: RallySegment[] = [
  { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null, metadata: {
    shot_count: 9, shot_count_confidence: "High" } },
  { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null, metadata: {
    shot_count: 5 } },
  { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null, metadata: {
    shot_count: 19, shot_count_confidence: "Medium" } },
  { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null, metadata:
    {} },
  { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null, metadata:
    { shot_count: 11 } },
  { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null, metadata:
    { shot_count: 24, shot_count_confidence: "High" } },
];

===LOCALE SIZES===
  88 src/i18n/locales/en/common.json
  88 src/i18n/locales/hi/common.json
  88 src/i18n/locales/kn/common.json
  264 total
===MAIN.tsx===
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "../App";
import "../i18n"; // initialize the i18next singleton before the app renders (tracker B1)
// Self-hosted OFL Noto fonts so Kannada/Hindi render crisply offline (tracker B4 / decision D7)
?
// no Google Fonts runtime dependency. unicode-range-scoped, so only Indic codepoints use them.
import "@fontsource-variable/noto-sans-kannada/index.css";

```

```
import "@fontsource-variable/noto-sans-devanagari/index.css";
import "../index.css";

const root = document.getElementById("root");
if (!root) throw new Error("#root not found");

createRoot(root).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

37. assistant (2026-06-21T13:36:11.113Z)

Now I have the full picture. Key confirmations:

- The 3 locale catalogs (en/kn/hi) are JSON files `import`ed statically into the i18n singleton, bundled into the single JS chunk at build time ? they ship inside the SPA bundle, NOT as separate runtime files. Fonts (Noto Kannada/Devanagari) are self-hosted via `@fontsource-variable`, also bundled (the woff2 files in `dist/assets`). No Google Fonts runtime dependency ? works offline.
- pyproject.toml `[tool.hatch.build.targets.wheel] packages = ["backend", "training"]` ? there's NO `force-include` or package-data directive, so even the existing `backend/api/static` legacy UI is likely NOT in the wheel. Let me verify whether hatchling includes the static files, and check the App.tsx for how it reads `?video=` (URL param) to confirm same-origin assumptions. Let me also check whether the engine has any DESKTOP_APP_PLAN or appliance doc and how it reads `?video`.

38. user (2026-06-21T13:36:13.796Z)

```
src/App.tsx:30:// Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
src/App.tsx:49: // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ?
seeded from the URL,
src/App.tsx:51: // without a reload (it also rewrites ?video= so a refresh is stable).
src/App.tsx:53: () => new URLSearchParams(window.location.search).get("video"),
src/App.tsx:235: window.history.replaceState(null, "",
`?video=${encodeURIComponent(id)}`);
src/components/IngestPanel.tsx:4:// rally-selection back-half (App lifts ?video=<id>). Honest by
construction: the engine reports
src/i18n/locales/en/common.json:19: "demo": "Demo data ? open with
<code>?video=?video_id</code> against a running engine to load a real session.",
src/i18n/locales/en/common.json:65: "noVideo": "Open with ?video=<video_id> against a running
engine to build a reel ? demo data can't be compiled."
src/i18n/locales/hi/common.json:19: "demo": "???? ???? ? ???? ???? ??? ???? ?? ??? ???? ????
?? ??? <code>?video=?video_id</code> ?? ??????",
src/i18n/locales/hi/common.json:65: "noVideo": "??? ????? ?? ??? ???? ???? ?? ???
?video=<video_id> ?? ????? ? ???? ???? ?????? ???? ?? ??????"
===IngestPanel relevant===
2:// `gs://` bucket URI or a path on the engine host ? and it submits to POST /api/process,
shows
3:// live progress while polling GET /api/jobs/{id}, and hands the resulting video_id to the
existing
```

39. user (2026-06-21T13:36:16.547Z)

```
===MANIFEST exists?===
ls: cannot access 'MANIFEST.in': No such file or directory
===any built wheel?===
ls: cannot access 'dist/': No such file or directory
===backend/main static dirs in tree===
app.js
index.html
style.css
```

40. assistant (2026-06-21T13:36:21.807Z)

I have everything I need. Let me do a final confirmation on whether hatchling's default would include `backend/api/static/\*` in the wheel (non-`.py` files under a package dir). Hatchling's wheel target by default includes all files under the listed package directories, including data files ? so `backend/api/static/\*.html/.js/.css` would be packaged. But the runtime path is a hardcoded relative `"backend/api/static/index.html"` resolved against CWD, which breaks in an installed-wheel context. Let me confirm that the path resolution is CWD-relative (not package-relative).

41. user (2026-06-21T13:36:23.095Z)

```
630         if not args.video_path:
631             print("[ERROR] Please provide --video-path to debug a specific clip.")
632             return
633         run_cli_diagnose_clip(args.video_path, args.debug_clip)
634         return
635
636     # Initialize Global Config and DB
637     global global_config, db_instance
638     global_config = load_app_config() # returns typed AppConfig
639     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
640
641     # The CLI --output-dir overrides the configured output directory. It now lives
642     # on the typed model (OutputConfig.output_dir), so every consumer ? including
643     # /api/compile ? reads the same value instead of a write-only runtime hack.
644     global_config.output.output_dir = args.output_dir
645     os.makedirs(global_config.output.output_dir, exist_ok=True)
646
647     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
648     db_instance = Database(db_path)
649
```

42. assistant (2026-06-21T13:36:27.480Z)

Confirmed. The static-serving paths in main.py (`"backend/api/static/index.html"`, `os.makedirs("backend/api/static")`, `StaticFiles(directory="backend/api/static")`) are all CWD-relative string literals ? they only work when the server is launched from the repo root, which breaks in a `pip install`ed wheel run from an arbitrary directory. This is a real packaging defect for the "downloadable app" goal.

I now have all the facts needed. Let me compile the structured output.

43. user (2026-06-21T13:37:26.618Z)

Structured output provided successfully

44. assistant (2026-06-21T13:37:33.943Z)

I've completed the investigation and returned my findings via StructuredOutput. Summary of what I found on bundling the web UI:

The operator SPA (Vite/React 19/TS/Tailwind v4) lives in the sibling ****khelsutra/web**** repo, builds to `khelsutra/web/dist/` (gitignored, ~669 KB static bundle), and talks to the engine via a hardcoded same-origin `const BASE = "/api"` (`web/src/api/client.ts:7`) ? no absolute URL, no consumed env var. That means shipping the built bundle behind FastAPI on one origin needs ****zero SPA code change****.

The engine today serves a **separate, legacy** vanilla-JS UI at `/` and `/static` (`backend/main.py:64-74`) ? live-mounted, not dead, but superseded by the khelsutra SPA and using CWD-relative paths that break in an installed wheel. The i18n catalogs (en/kn/hi) and Indic fonts are compiled into the bundle (offline-ready, no runtime fetch). The main packaging

work is a cross-repo build coupling (``npm run build`` in `khelsutra ? vendor `dist/`` into the wheel via a hatchling ``force-include``, currently absent) plus fixing the engine's CWD-relative static paths and an SPA history-fallback route.